1. Set Up a Google Cloud Project:

- **Create a Project:**
  - Navigate to the Google Cloud Console.
  - Click on the project dropdown and select "New Project."
  - Provide a name for your project and create it.
- **Enable the Google Drive API:**
  - Within your project dashboard, go to "APIs & Services" > "Library."
  - Search for "Google Drive API" and click "Enable."
- **Create OAuth 2.0 Credentials:**
  - Navigate to "APIs & Services" > "Credentials."
  - Click on "Create Credentials" and choose "OAuth client ID."
  - Configure the consent screen if prompted.
  - Select "Desktop app" as the application type and create the credentials.
  - Download the `credentials.json` file and save it in your project directory.

Set Up the Node.js Environment:

- **Initialize the Project:**
  - Ensure you have Node.js installed.
  - Initialize your project:

```
npm init -y
```

Install Required Packages:

- Install the Google APIs client library:

```
npm install googleapis@105 @google-cloud/local-auth@2.1.0
```

Implement the Ownership Transfer Script:

- **Create the Script File:**
  - Create a file named `index.js` in your project directory.
- **Add the Following Code to `index.js`:**

```
const fs = require("fs").promises;

const path = require("path");

const { authenticate } = require("@google-cloud/local-auth");

const { google } = require("googleapis");



// Scopes required for the Drive API

const SCOPES = ["https://www.googleapis.com/auth/drive"];



// Path to the credentials file

const CREDENTIALS_PATH = path.join(__dirname, "credentials.json");



async function main() {

  // Authenticate the user

  const auth = await authenticate({

    keyfilePath: CREDENTIALS_PATH,

    scopes: SCOPES,

  });



  const drive = google.drive({ version: "v3", auth });



  // ID of the file to transfer ownership

  const fileId = "YOUR_FILE_ID";

  // Email of the new owner

  const newOwnerEmail = "newowner@example.com";



  try {
```

```
// Check if the new owner already has permissions
const res = await drive.permissions.list({
  fileId: fileId,
  fields: "permissions(id, emailAddress)",
});

const permissions = res.data.permissions;
let permissionId = null;

// Find the permission ID for the new owner
for (const permission of permissions) {
  if (permission.emailAddress === newOwnerEmail) {
    permissionId = permission.id;
    break;
  }
}

if (!permissionId) {
  // If the new owner doesn't have permissions, create one
  const res = await drive.permissions.create({
    fileId: fileId,
    requestBody: {
      role: "writer",
      type: "user",
      emailAddress: newOwnerEmail,
    },
```

```
        fields: "id",
      });
      permissionId = res.data.id;
    }

    // Transfer ownership
    await drive.permissions.update({
      fileId: fileId,
      permissionId: permissionId,
      transferOwnership: true,
      requestBody: {
        role: "owner",
      },
    });

    console.log(`Ownership of file ${fileId} transferred to ${newOwnerEmail}.`);
  } catch (error) {
    console.error("Error transferring ownership:", error);
  }
}

main().catch(console.error);
```

Explanation:

- **Authentication:** The script uses OAuth2 to authenticate the user. Ensure that the authenticated user is the current owner of the file.
- **Permission Check:** It checks if the new owner already has permissions for the file. If not, it grants 'writer' permission to the new owner.
- **Ownership Transfer:** The script then updates the permission to transfer ownership to the new user.

Run the Script:

- Execute the script using Node.js:

```
node index.js
```

Important Considerations:

- **Domain Restrictions:** Transferring ownership is typically allowed within the same Google Workspace domain. Transferring ownership outside the domain may not be permitted.
- **Shared Drives:** Ownership transfers are not supported for files and folders in shared drives.
- **Permissions:** After the transfer, the previous owner's role is downgraded to 'writer'.